Import Libraries

In [90]:
```
# Part 1 : Data Import & Inspection
# California Housing Prices Dataset

import pandas as pd
import numpy as np

import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import ( mean_absolute_error, mean_squared_error,
r2_score)
```

Load Dataset

In [110]:
```
# Load the California Housing dataset

df = pd.read_csv("housing.csv")

print("Dataset loaded successfully.")
```

```
Dataset loaded successfully.
```

In [92]:
```
df.columns
```

Out[92]:
```
Index(['longitude', 'latitude', 'housing_median_age',
'total_rooms',
       'total_bedrooms', 'population', 'households',
'median_income',
       'median_house_value', 'ocean_proximity'],
      dtype='object')
```

Display Dataset

In [93]:
```
display(df.head(10))

display(df.tail())
```

	longitude	latitude	housing_median_age	total_rooms	total_bedr
0	-122.23	37.88	41.0	880.0	129.0
1	-122.22	37.86	21.0	7099.0	1106.0
2	-122.24	37.85	52.0	1467.0	190.0
3	-122.25	37.85	52.0	1274.0	235.0
4	-122.25	37.85	52.0	1627.0	280.0
5	-122.25	37.85	52.0	919.0	213.0
6	-122.25	37.84	52.0	2535.0	489.0
7	-122.25	37.84	52.0	3104.0	687.0
8	-122.26	37.84	42.0	2555.0	665.0
9	-122.25	37.84	52.0	3549.0	707.0

	longitude	latitude	housing_median_age	total_rooms	total_
20635	-121.09	39.48	25.0	1665.0	374.0
20636	-121.21	39.49	18.0	697.0	150.0
20637	-121.22	39.43	17.0	2254.0	485.0
20638	-121.32	39.43	18.0	1860.0	409.0
20639	-121.24	39.37	16.0	2785.0	616.0

`

Observation

The first and last few rows help verify that the dataset has been loaded correctly.

We can also get an initial understanding of the available features and the types of values stored in each column.

`

Dataset Shape

In [111]:
```
rows, columns = df.shape

print(f"Number of Rows    : {rows}")
print(f"Number of Columns : {columns}")

print(f"\nDataset Shape = ({rows}, {columns})")
```

```
Number of Rows    : 20640
Number of Columns : 10

Dataset Shape = (20640, 10)
```

`

Observation

The dataset contains **20,640 rows** and **10 columns**.

Each row represents a housing district in California, while each column describes one characteristic of that district.

`

Data Types

In [112]:
```
print("Data Types")

display(df.dtypes)
```

```
Data Types
```

	0
longitude	float64
latitude	float64
housing_median_age	float64
total_rooms	float64
total_bedrooms	float64
population	float64
households	float64
median_income	float64
median_house_value	float64
ocean_proximity	object

dtype: object

Dataset Information

In [113]:
```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 20640 entries, 0 to 20639
Data columns (total 10 columns):
 #   Column              Non-Null Count  Dtype  
---  ------              --------------  -----  
 0   longitude           20640 non-null  float64
 1   latitude            20640 non-null  float64
 2   housing_median_age  20640 non-null  float64
 3   total_rooms         20640 non-null  float64
 4   total_bedrooms      20433 non-null  float64
 5   population          20640 non-null  float64
 6   households          20640 non-null  float64
 7   median_income       20640 non-null  float64
 8   median_house_value  20640 non-null  float64
 9   ocean_proximity     20640 non-null  object 
dtypes: float64(9), object(1)
memory usage: 1.6+ MB
```

`

Observation

The `info()` method provides useful information such as:

- Total number of entries
- Number of columns
- Data type of every feature
- Number of non-null values
- Memory usage

This helps identify missing values and incorrect data types.

`

Numerical and Categorical Columns

In [114]:
```
numerical_columns = df.select_dtypes(include=
["number"]).columns.tolist()

categorical_columns = df.select_dtypes(include=
["object"]).columns.tolist()

print("------------Numerical Features-----------")

for column in numerical_columns:
    print(column)

print("\n-----------Categorical Features------------")

for column in categorical_columns:
    print(column)
```

```
------------Numerical Features-----------
longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income
median_house_value

-----------Categorical Features------------
ocean_proximity
```

Descriptive Statistics

In [115]:
```
display(df.describe())
```

	longitude	latitude	housing_median_age	total_roo
count	20640.000000	20640.000000	20640.000000	20640.0000
mean	-119.569704	35.631861	28.639486	2635.76308
std	2.003532	2.135952	12.585558	2181.61525
min	-124.350000	32.540000	1.000000	2.000000
25%	-121.800000	33.930000	18.000000	1447.75000
50%	-118.490000	34.260000	29.000000	2127.00000
75%	-118.010000	37.710000	37.000000	3148.00000
max	-114.310000	41.950000	52.000000	39320.0000

`

Observation

- The dataset contains **20,640 rows** and **9 numerical features**.
- The `total_bedrooms` column has **207 missing values** and will need cleaning.
- The average house value is approximately **$206,856**, with values ranging from **$14,999** to **$500,001**.
- Features like **total_rooms**, **population**, and **households** have a wide range of values, indicating possible outliers.
- The **median_income** feature also varies significantly and may strongly influence house prices.
- Most numerical features appear to be **right-skewed** since their mean is greater than their median.
- The dataset includes geographical features (**longitude** and **latitude**) that represent locations across California.
- Overall, the dataset is well-structured and ready for the data cleaning process.

`

Feature Explanation

`

Feature Description

Feature	Meaning
longitude	Geographic longitude of the housing district.
latitude	Geographic latitude of the housing district.
housing_median_age	Median age of houses in the district (years).
total_rooms	Total number of rooms in all houses within the district.
total_bedrooms	Total number of bedrooms in the district.
population	Total population living in the district.
households	Total number of households in the district.
median_income	Median household income (measured in tens of thousands of US dollars).
median_house_value	Median house price in the district. This is the **target variable** that we will predict later using regression models.
ocean_proximity	Categorical feature indicating how close the district is to the Pacific Ocean (e.g., NEAR BAY, INLAND, NEAR OCEAN).

`

Part 2 – Data Cleaning

Check Missing Values

In [116]:
```
missing_values = df.isnull().sum()

display(missing_values)
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	207
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

dtype: int64

Observation

- Most columns do not contain missing values.
- The **total_bedrooms** column contains **207 missing values**.
- Missing values must be handled before further analysis because many machine learning algorithms cannot work with missing data.

Missing Value Percentage

In [117]:
```
missing_percentage = (df.isnull().sum() / len(df)) * 100

missing_df = pd.DataFrame({
    "Missing Values": missing_values,
    "Percentage (%)": missing_percentage.round(2)
})

display(missing_df)
```

	Missing Values	Percentage (%)
longitude	0	0.0
latitude	0	0.0
housing_median_age	0	0.0
total_rooms	0	0.0
total_bedrooms	207	1.0
population	0	0.0
households	0	0.0
median_income	0	0.0
median_house_value	0	0.0
ocean_proximity	0	0.0

Handling Missing Values

The `total_bedrooms` column contains **207 missing values** (about **1%** of the dataset).

The missing values were filled using the **median** because it is less affected by outliers and is more suitable for skewed numerical data.

Why not use other methods?

- **Mean:** Can be influenced by extreme values, making the filled values less representative.
- **Mode:** More suitable for categorical data, not continuous numerical features.
- **Drop Rows:** Would remove useful data unnecessarily since only a small percentage of values are missing.

Decision: The missing values were replaced with the **median** to preserve all records while maintaining data quality.

Fill Missing Values

In [118]:
```
median_bedrooms = df["total_bedrooms"].median()

df["total_bedrooms"].fillna(median_bedrooms, inplace=True)

print("Missing values filled successfully.")
```

```
Missing values filled successfully.
```

```
/tmp/ipykernel_754/1193899318.py:3: FutureWarning: A value is
trying to be set on a copy of a DataFrame or Series through
chained assignment using an inplace method.
The behavior will change in pandas 3.0. This inplace method
will never work because the intermediate object on which we are
setting values always behaves as a copy.

For example, when doing 'df[col].method(value, inplace=True)',
try using 'df.method({col: value}, inplace=True)' or df[col] =
df[col].method(value) instead, to perform the operation inplace
on the original object.


  df["total_bedrooms"].fillna(median_bedrooms, inplace=True)
```

Verify Missing Values

In [25]:
```
display(df.isnull().sum())
```

	0
longitude	0
latitude	0
housing_median_age	0
total_rooms	0
total_bedrooms	0
population	0
households	0
median_income	0
median_house_value	0
ocean_proximity	0

dtype: int64

Observation

All missing values have been successfully removed from the dataset.

Check Duplicate Rows

In [26]:
```
duplicate_rows = df.duplicated().sum()

print(f"Number of Duplicate Rows : {duplicate_rows}")
```

```
Number of Duplicate Rows : 0
```

Markdown

No duplicates row are present in our dataset.

Check Incorrect / Impossible Values

In [ ]:
```
print("---------Checking for Incorrect Values-----------")


print("Negative Housing Age:",
      (df["housing_median_age"] < 0).sum())

print("Zero or Negative House Price:",
      (df["median_house_value"] <= 0).sum())

print("Negative Population:",
      (df["population"] < 0).sum())

print("Negative Total Rooms:",
      (df["total_rooms"] < 0).sum())

print("Negative Total Bedrooms:",
      (df["total_bedrooms"] < 0).sum())

print("Negative Households:",
      (df["households"] < 0).sum())

print("Negative Median Income:",
      (df["median_income"] < 0).sum())
```

```
---------Checking for Incorrect Values-----------
Negative Housing Age: 0
Zero or Negative House Price: 0
Negative Population: 0
Negative Total Rooms: 0
Negative Total Bedrooms: 0
Negative Households: 0
Negative Median Income: 0
```

Observation

No incorrect or impossible values were found.

- No negative housing age.
- No negative population.
- No negative room counts.
- No zero or negative house prices.
- No negative income values.

The dataset appears to be logically consistent.

Verify Data Types

In [ ]:
```
display(df.dtypes)
```

	0
longitude	float64
latitude	float64
housing_median_age	float64
total_rooms	float64
total_bedrooms	float64
population	float64
households	float64
median_income	float64
median_house_value	float64
ocean_proximity	object

dtype: object

Final Dataset Shape

In [ ]:
```
rows, columns = df.shape

print(f"Rows    : {rows}")
print(f"Columns : {columns}")
```

```
Rows    : 20640
Columns : 10
```

Data Cleaning Summary

The following cleaning steps were completed successfully:

- Checked all columns for missing values.
- Filled missing values in **total_bedrooms** using the median.
- Verified that no missing values remain.
- Checked for duplicate records (none found).
- Verified that there are no incorrect or impossible values.
- Confirmed that all columns have the correct data types.

The dataset is now clean and ready for **Exploratory Data Analysis (EDA)** and machine learning.

In [ ]:
```

```

Part 3 — Exploratory Data Analysis (EDA)

In [ ]:

```
numerical_columns = df.select_dtypes(include=["number"]).columns

print("Numerical Features----------")

for column in numerical_columns:
    print("-", column)
```

```
Numerical Features----------
- longitude
- latitude
- housing_median_age
- total_rooms
- total_bedrooms
- population
- households
- median_income
- median_house_value
```

3.1 Summary Statistics

In [ ]:
```
summary_statistics = pd.DataFrame(index=numerical_columns)

# Central Tendency
summary_statistics["Mean"] = df[numerical_columns].mean()
summary_statistics["Median"] = df[numerical_columns].median()
summary_statistics["Mode"] = df[numerical_columns].mode().iloc[0]

# Spread
summary_statistics["Variance"] = df[numerical_columns].var()
summary_statistics["Standard Deviation"] = df[numerical_columns].std()

# Quartiles
summary_statistics["Q1"] = df[numerical_columns].quantile(0.25)
summary_statistics["Q3"] = df[numerical_columns].quantile(0.75)

# Interquartile Range
summary_statistics["IQR"] = (
    summary_statistics["Q3"] - summary_statistics["Q1"]
)

# Skewness
summary_statistics["Skewness"] = df[numerical_columns].skew()

# Round values for readability
summary_statistics = summary_statistics.round(2)

display(summary_statistics)
```

	Mean	Median	Mode	Variance
longitude	-119.57	-118.49	-118.31	4.010000e+00
latitude	35.63	34.26	34.06	4.560000e+00
housing_median_age	28.64	29.00	52.00	1.584000e+02
total_rooms	2635.76	2127.00	1527.00	4.759445e+06
total_bedrooms	536.84	435.00	435.00	1.758895e+05
population	1425.48	1166.00	891.00	1.282470e+06
households	499.54	409.00	306.00	1.461760e+05
median_income	3.87	3.53	3.12	3.610000e+00
median_house_value	206855.82	179700.00	500001.00	1.331615e+10

Observations

- **Longitude:** Slight negative skew (skewness = -0.30). Mean is close to the median, indicating a fairly balanced distribution.
- **Latitude:** Mild positive skew (0.47). Mean is slightly higher than the median.
- **Housing Median Age:** Nearly symmetric distribution (skewness = 0.06). Mean and median are almost equal.
- **Total Rooms:** Highly positively skewed (4.15), indicating the presence of large outliers.
- **Total Bedrooms:** Strong positive skew (3.48) with noticeable high-value outliers.
- **Population:** Highest positive skew (4.94), showing extreme outliers and high variability.
- **Households:** Strong positive skew (3.41), indicating some districts have much larger household counts.
- **Median Income:** Moderately positively skewed (1.65), with a few high-income districts.
- **Median House Value:** Moderately positively skewed (0.98). The mode is **500001**, suggesting many values reached the dataset's maximum price limit.

Overall

- Most numerical features are **positively skewed**, especially **Population**, **Total Rooms**, **Total Bedrooms**, and **Households**, indicating outliers.
- **Housing Median Age** is the most evenly distributed feature.
- Count-based features have much higher variability than geographical features.

Identify Skewed Features

In [ ]:
```
skew_result = []

for column in numerical_columns:

    skew = df[column].skew()

    if skew > 0.5:
        distribution = "Right Skewed"

    elif skew < -0.5:
        distribution = "Left Skewed"

    else:
        distribution = "Approximately Symmetric"

    skew_result.append({
        "Feature": column,
        "Skewness": round(skew, 2),
        "Distribution": distribution
    })

skew_df = pd.DataFrame(skew_result)

display(skew_df)
```

	Feature	Skewness	Distribution
0	longitude	-0.30	Approximately Symmetric
1	latitude	0.47	Approximately Symmetric
2	housing_median_age	0.06	Approximately Symmetric
3	total_rooms	4.15	Right Skewed
4	total_bedrooms	3.48	Right Skewed
5	population	4.94	Right Skewed
6	households	3.41	Right Skewed
7	median_income	1.65	Right Skewed
8	median_house_value	0.98	Right Skewed

Observation

- Features with **positive skewness** have a longer tail on the right side.
- Features with **negative skewness** have a longer tail on the left side.
- Features with skewness close to **0** are approximately symmetric.

Highly skewed features may require transformation during feature engineering.

Compare Mean and Median

In [ ]:
```
comparison = pd.DataFrame({
    "Mean": df[numerical_columns].mean().round(2),
    "Median": df[numerical_columns].median().round(2)
})

comparison["Difference"] = (
    comparison["Mean"] - comparison["Median"]
).round(2)

comparison["Skewness"] = df[numerical_columns].skew().round(2)

display(comparison)
```

	Mean	Median	Difference	Skewness
longitude	-119.57	-118.49	-1.08	-0.30
latitude	35.63	34.26	1.37	0.47
housing_median_age	28.64	29.00	-0.36	0.06
total_rooms	2635.76	2127.00	508.76	4.15
total_bedrooms	536.84	435.00	101.84	3.48
population	1425.48	1166.00	259.48	4.94
households	499.54	409.00	90.54	3.41
median_income	3.87	3.53	0.34	1.65
median_house_value	206855.82	179700.00	27155.82	0.98

Observations

By comparing the **mean** and **median**, we can identify skewed features.

- **Longitude:** Slight negative skew, as the mean is lower than the median.
- **Latitude:** Mild positive skew, with the mean slightly higher than the median.
- **Housing Median Age:** Nearly symmetric since the mean and median are almost equal.
- **Total Rooms, Total Bedrooms, Population, and Households:** Strong positive skew, indicated by large mean–median differences and high skewness values.
- **Median Income:** Moderately positively skewed, with some higher-income districts increasing the mean.
- **Median House Value:** Moderately positively skewed, showing the presence of higher-priced houses.

Overall

- Most features are **positively skewed**, indicating the presence of high-value outliers.
- **Housing Median Age** is the closest to a normal (symmetric) distribution.
- **Population** has the highest skewness (**4.94**), making it the most skewed feature.

Highlight Highly Skewed Features

In [ ]:
```
highly_skewed = summary_statistics[
    summary_statistics["Skewness"].abs() > 1
]

print("Highly Skewed Features:\n")

display(highly_skewed[["Skewness"]])
```

```
Highly Skewed Features:
```

	Skewness
total_rooms	4.15
total_bedrooms	3.48
population	4.94
households	3.41
median_income	1.65

Observation

Features with an absolute skewness greater than **1** are considered highly skewed.

These features may contain extreme values or outliers and can affect machine learning models. They may benefit from transformations such as logarithmic scaling in future analysis.

In [ ]:
```
display(summary_statistics)
```

	Mean	Median	Mode	Variance
longitude	-119.57	-118.49	-118.31	4.010000e+00
latitude	35.63	34.26	34.06	4.560000e+00
housing_median_age	28.64	29.00	52.00	1.584000e+02
total_rooms	2635.76	2127.00	1527.00	4.759445e+06
total_bedrooms	536.84	435.00	435.00	1.758895e+05
population	1425.48	1166.00	891.00	1.282470e+06
households	499.54	409.00	306.00	1.461760e+05
median_income	3.87	3.53	3.12	3.610000e+00
median_house_value	206855.82	179700.00	500001.00	1.331615e+10

Summary of Statistical Analysis

From the summary statistics, we observed that:

- Some features have a much larger spread than others, indicating high variability.
- Several features are positively skewed, suggesting the presence of extreme values.
- The difference between the mean and median confirms that many numerical features are not normally distributed.
- Features with high standard deviation and IQR have greater variation across housing districts.
- These findings will help us identify outliers and better understand the data during visualization in the next section.

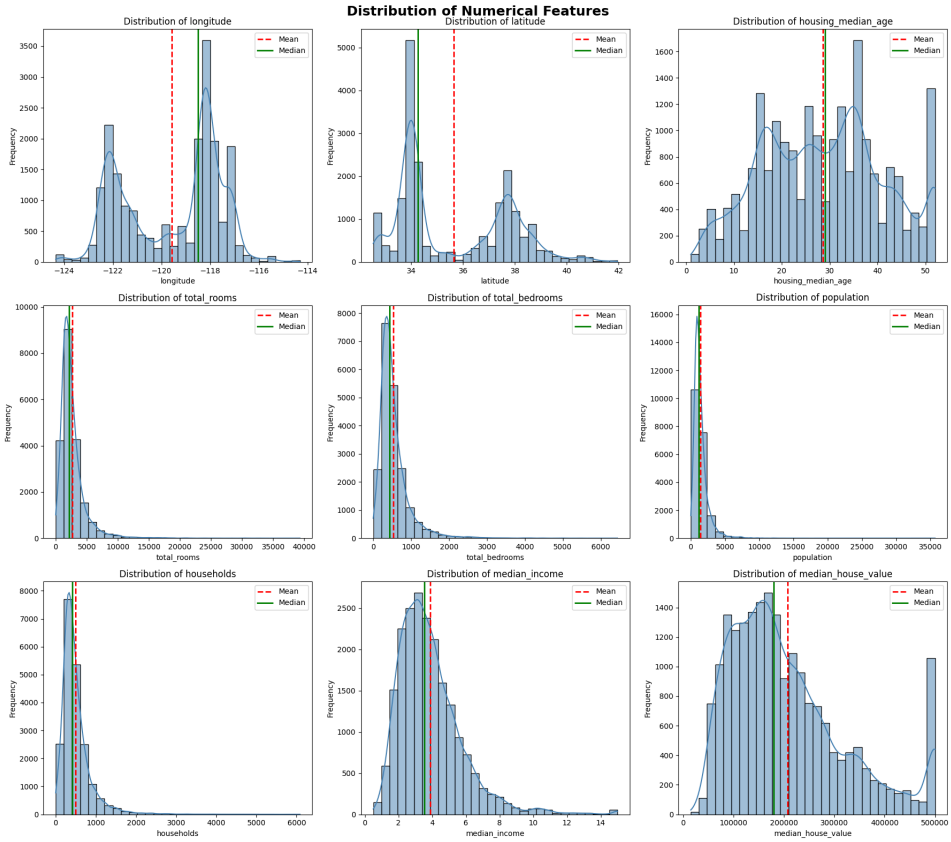
3.2 Visualizations

1. Histograms

Histograms show how the values of each numerical feature are distributed. They help identify whether the data is normally distributed, skewed, or contains multiple peaks.

In [ ]:

```
# Histograms for All Numerical Features

numerical_columns = df.select_dtypes(include=["number"]).columns

plt.figure(figsize=(18, 16))

for i, column in enumerate(numerical_columns):

    plt.subplot(3, 3, i + 1)

    sns.histplot(
        data=df,
        x=column,
        bins=30,
        kde=True,
        color="steelblue",
        edgecolor="black"
    )

    # Mean
    plt.axvline(
        df[column].mean(),
        color="red",
        linestyle="--",
        linewidth=2,
        label="Mean"
    )

    # Median
    plt.axvline(
        df[column].median(),
        color="green",
        linestyle="-",
        linewidth=2,
        label="Median"
    )

    plt.title(f"Distribution of {column}", fontsize=12)
    plt.xlabel(column)
    plt.ylabel("Frequency")
    plt.legend()

plt.suptitle(
    "Distribution of Numerical Features",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```


Distribution of Numerical Features
Distribution of longitude
Distribution of latitude
Distribution of housing_median_age
Distribution of total_rooms
Distribution of total_bedrooms
Distribution of population
Distribution of households
Distribution of median_income
Distribution of median_house_value
Mean
Median
Frequency
longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income
median_house_value

3.2 Histogram Observations

1. Longitude

- The distribution is **slightly negatively skewed** (skewness = -0.30).
- The mean (-119.57) lies to the left of the median (-118.49), confirming the negative skew.
- Two distinct peaks indicate that houses are concentrated in two major longitude regions.

---

2. Latitude

- The distribution is **slightly positively skewed** (skewness = 0.47).
- The mean (35.63) is slightly higher than the median (34.26).
- The histogram shows two main clusters, suggesting houses are concentrated around specific latitude ranges.

---

3. Housing Median Age

- The distribution is **nearly symmetric** (skewness = 0.06).
- Mean (28.64) and median (29.00) are almost identical.
- A noticeable spike at **52 years** indicates that many houses have reached the dataset's maximum recorded age.

---

4. Total Rooms

- The distribution is **highly positively skewed** (skewness = 4.15).
- Mean (2635.76) is much higher than the median (2127), indicating the presence of large values.
- Most districts have relatively few rooms, while a small number have extremely large room counts, creating a long right tail.

---

5. Total Bedrooms

- The distribution is **strongly right-skewed** (skewness = 3.48).
- Mean (536.84) is higher than the median (435).
- Most observations are concentrated at lower bedroom counts, with several extreme values acting as outliers.

---

6. Population

- Population is the **most positively skewed** feature (skewness = 4.94).
- Mean (1425.48) is considerably larger than the median (1166).
- Most districts have moderate populations, while a few districts have very high populations, producing a long right tail.

---

7. Households

- The distribution is **strongly positively skewed** (skewness = 3.41).
- Mean (499.54) exceeds the median (409).
- Most districts contain relatively few households, while a small number contain exceptionally high household counts.

---

8. Median Income

- The distribution is **moderately positively skewed** (skewness = 1.65).
- Mean (3.87) is slightly higher than the median (3.53).
- Most districts have median incomes between **2 and 5**, while fewer districts have very high incomes.

---

9. Median House Value

- The distribution is **moderately positively skewed** (skewness = 0.98).
- Mean ($206,856) is higher than the median ($179,700).
- Most house values lie between **$100,000 and $250,000**.
- A large spike at **$500,001** indicates that house prices were capped at the dataset's maximum value.

Overall Observations

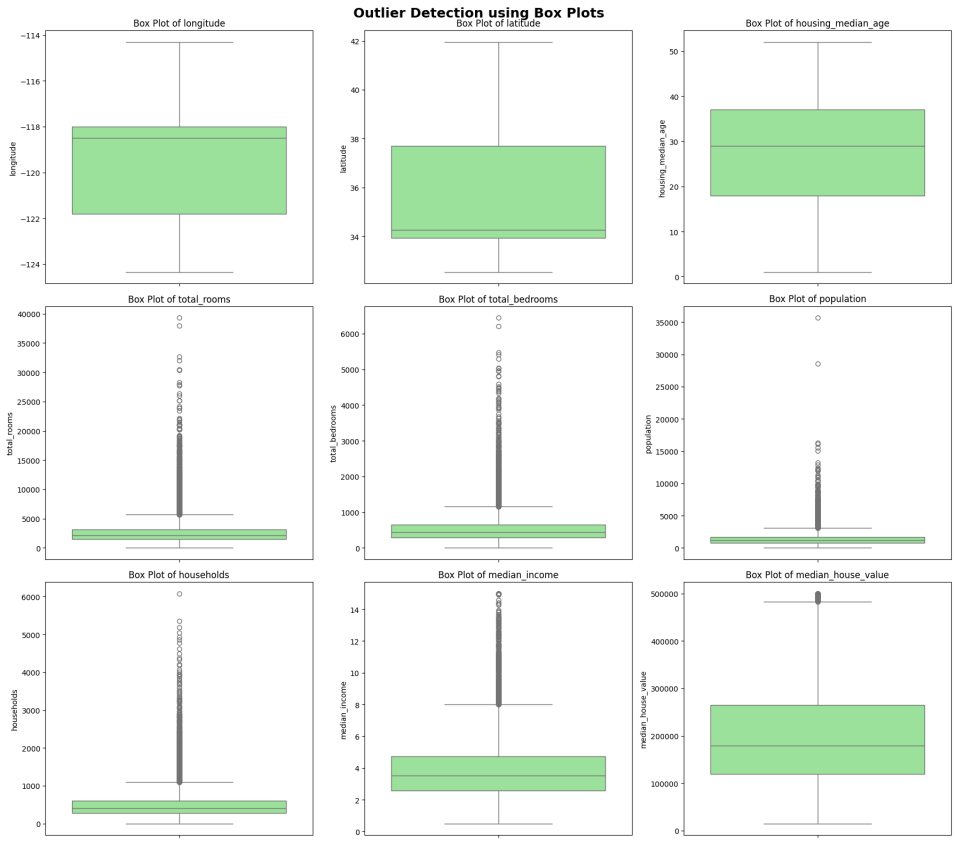
- Most numerical features are **positively skewed**, indicating the presence of high-value outliers.
- **Population**, **Total Rooms**, **Total Bedrooms**, and **Households** show the strongest positive skew and greatest variability.
- **Housing Median Age** is the closest to a symmetric distribution.
- **Longitude** is the only feature with a slight negative skew.
- The spikes at **52 years** (Housing Median Age) and **$500,001** (Median House Value) suggest that these variables have upper limits in the dataset.

2. Box Plots

Box plots summarize the distribution of each feature and highlight potential outliers using the Interquartile Range (IQR) method.

In [ ]:

```
# ============================================================
# Box Plots
# ============================================================

plt.figure(figsize=(18,16))

for i, column in enumerate(numerical_columns):

    plt.subplot(3,3,i+1)

    sns.boxplot(
        y=df[column],
        color="lightgreen"
    )

    plt.title(f"Box Plot of {column}")
    plt.ylabel(column)

plt.suptitle(
    "Outlier Detection using Box Plots",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```



Box Plot Observations

Longitude

- No significant outliers are visible.
- The spread is moderate and the distribution is fairly balanced.

Latitude

- No major outliers are present.
- Values are concentrated within a relatively narrow range.

Housing Median Age

- Very few outliers are observed.
- The distribution is fairly symmetric with moderate spread.

Total Rooms

- A large number of **upper outliers** are present.
- This indicates some districts have extremely high room counts.

Total Bedrooms

- Many **upper outliers** are visible.
- Bedroom counts vary greatly across districts.

Population

- Contains a very large number of **extreme upper outliers**.
- Population is the most variable feature.

Households

- Numerous **upper outliers** are present.
- Most districts have relatively low household counts, while a few have exceptionally high counts.

Median Income

- Several high-income outliers are visible.
- Most observations are concentrated in the lower income range.

Median House Value

- Many observations lie near the **upper limit (~$500,000)**.
- The clustering near the maximum value suggests the house prices are **capped** in the dataset.

Overall

- **Total Rooms, Total Bedrooms, Population, and Households** show the strongest outlier presence and highest variability.
- **Longitude, Latitude, and Housing Median Age** have comparatively stable distributions with minimal outliers.
- **Median House Value** shows a clear ceiling effect due to the dataset's maximum price cap.

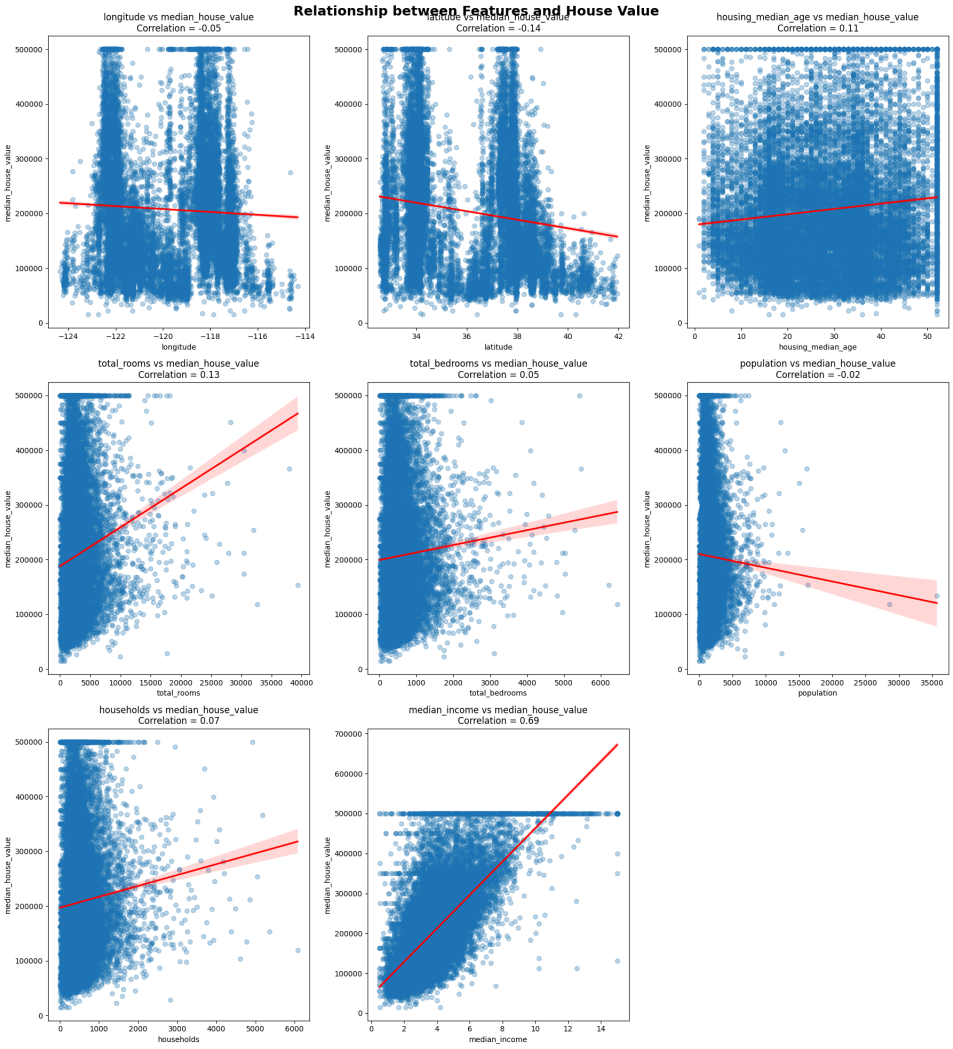
3. Scatter Plots

Scatter plots help visualize the relationship between each numerical feature and the target variable (`median_house_value`).

A regression line is added to better understand the overall trend.

In [ ]:

```
# ============================================================
# Scatter Plots with Regression Line
# ============================================================

target = "median_house_value"

features = numerical_columns.drop(target)

plt.figure(figsize=(18,20))

for i, column in enumerate(features):

    plt.subplot(3,3,i+1)

    sns.regplot(
        data=df,
        x=column,
        y=target,
        scatter_kws={"alpha":0.3},
        line_kws={"color":"red"}
    )

    corr = df[column].corr(df[target])

    plt.title(f"{column} vs {target}\nCorrelation = {corr:.2f}")

    plt.xlabel(column)
    plt.ylabel(target)

plt.suptitle(
    "Relationship between Features and House Value",
    fontsize=18,
    fontweight="bold"
)

plt.tight_layout()

plt.show()
```


Relationship between Features and House Value
longitude vs median_house_value
Correlation = -0.05
latitude vs median_house_value
Correlation = -0.14
housing_median_age vs median_house_value
Correlation = 0.11
total_rooms vs median_house_value
Correlation = 0.13
total_bedrooms vs median_house_value
Correlation = 0.05
population vs median_house_value
Correlation = -0.02
households vs median_house_value
Correlation = 0.07
median_income vs median_house_value
Correlation = 0.69
median_house_value
longitude
latitude
housing_median_age
total_rooms
total_bedrooms
population
households
median_income

Scatter Plot Observations

- **Longitude:** Very weak negative correlation (**-0.05**); little effect on house value.
- **Latitude:** Weak negative correlation (**-0.14**); house values decrease slightly with increasing latitude.
- **Housing Median Age:** Weak positive correlation (**0.11**); older houses tend to have slightly higher values.
- **Total Rooms:** Weak positive correlation (**0.13**); more rooms are associated with slightly higher house values.
- **Total Bedrooms:** Very weak positive correlation (**0.05**); minimal impact on house value.
- **Population:** Almost no relationship with house value (**-0.02**).
- **Households:** Very weak positive correlation (**0.07**); little influence on house value.
- **Median Income:** Strong positive correlation (**0.69**); higher income is associated with higher house values and is the strongest predictor.

Overall

- **Median income** has the strongest relationship with house value.
- All other features show **weak or negligible correlations**, indicating limited individual influence on house prices.
- The horizontal line at **$500,001** indicates that house values are capped in the dataset.

4. Correlation Heatmap

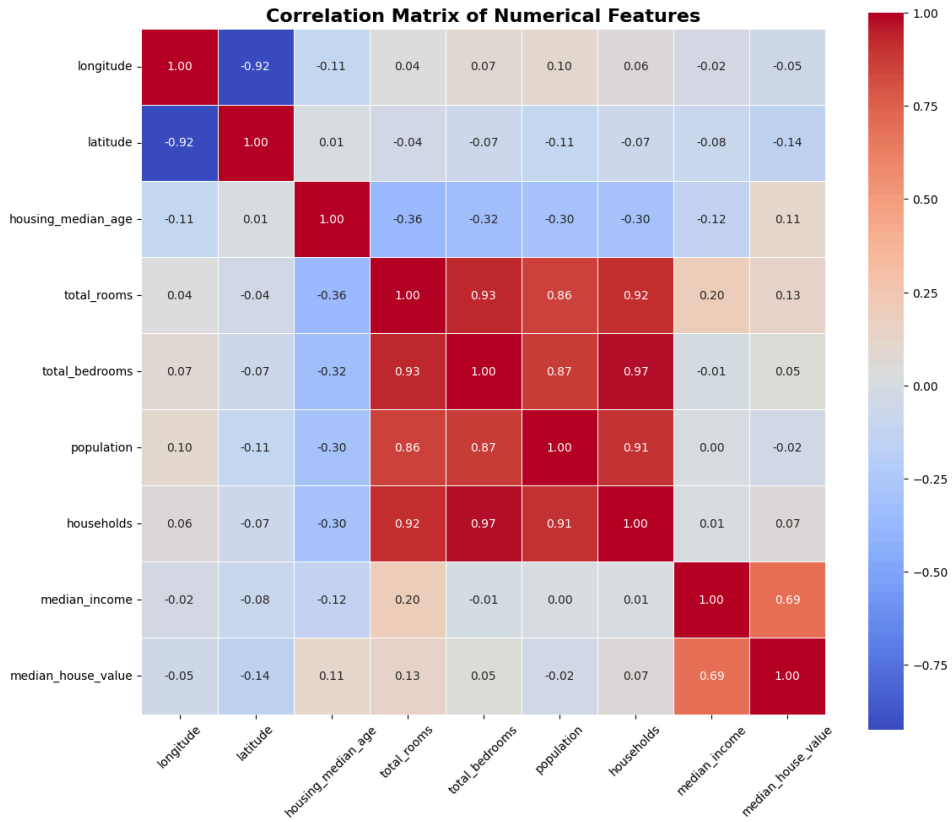
The heatmap shows the correlation between numerical features.

Correlation values range from **-1 to 1**:

- **1** → Strong positive relationship
- **0** → No relationship
- **-1** → Strong negative relationship

In [ ]:

```
# ============================================================
# Correlation Heatmap
# ============================================================

correlation = df[numerical_columns].corr()

plt.figure(figsize=(12,10))

sns.heatmap(
    correlation,
    annot=True,
    cmap="coolwarm",
    fmt=".2f",
    linewidths=0.5,
    square=True
)

plt.title(
    "Correlation Matrix of Numerical Features",
    fontsize=16,
    fontweight="bold"
)

plt.xticks(rotation=45)
plt.yticks(rotation=0)

plt.tight_layout()

plt.show()
```



Correlation Matrix Observations

- **Median Income** has the strongest positive correlation with **Median House Value** (**0.69**), indicating that areas with higher incomes generally have higher house prices. It is the most important feature for predicting house value.

- **Total Rooms, Total Bedrooms, Population, and Households** are strongly correlated with each other (**0.86–0.97**). This means these variables contain similar information, and using all of them together may introduce **multicollinearity** in predictive models.

- **Longitude** and **Latitude** have a strong negative correlation (**-0.92**), reflecting the geographical layout of the locations in the dataset.

- **Housing Median Age** has a moderate negative correlation with **Total Rooms (-0.36)**, **Total Bedrooms (-0.32)**, **Population (-0.30)**, and **Households (-0.30)**. This suggests that older neighborhoods generally have fewer rooms, fewer households, and lower populations.

- The remaining features have **very weak correlations** with **Median House Value** (between **-0.14** and **0.13**), indicating that individually they have limited influence on house prices.

Overall

- **Median Income** is the best predictor of house value.
- **Total Rooms, Total Bedrooms, Population, and Households** are highly related and may provide redundant information.
- Most other variables show only weak relationships with house prices, suggesting that house values are influenced by multiple factors rather than a single feature.

In [ ]:
```
# Sort correlations with target.

target_corr = (
    correlation["median_house_value"]
    .sort_values(ascending=False)
)

display(target_corr)
```

	median_house_value
median_house_value	1.000000
median_income	0.688075
total_rooms	0.134153
housing_median_age	0.105623
households	0.065843
total_bedrooms	0.049457
population	-0.024650
longitude	-0.045967
latitude	-0.144160

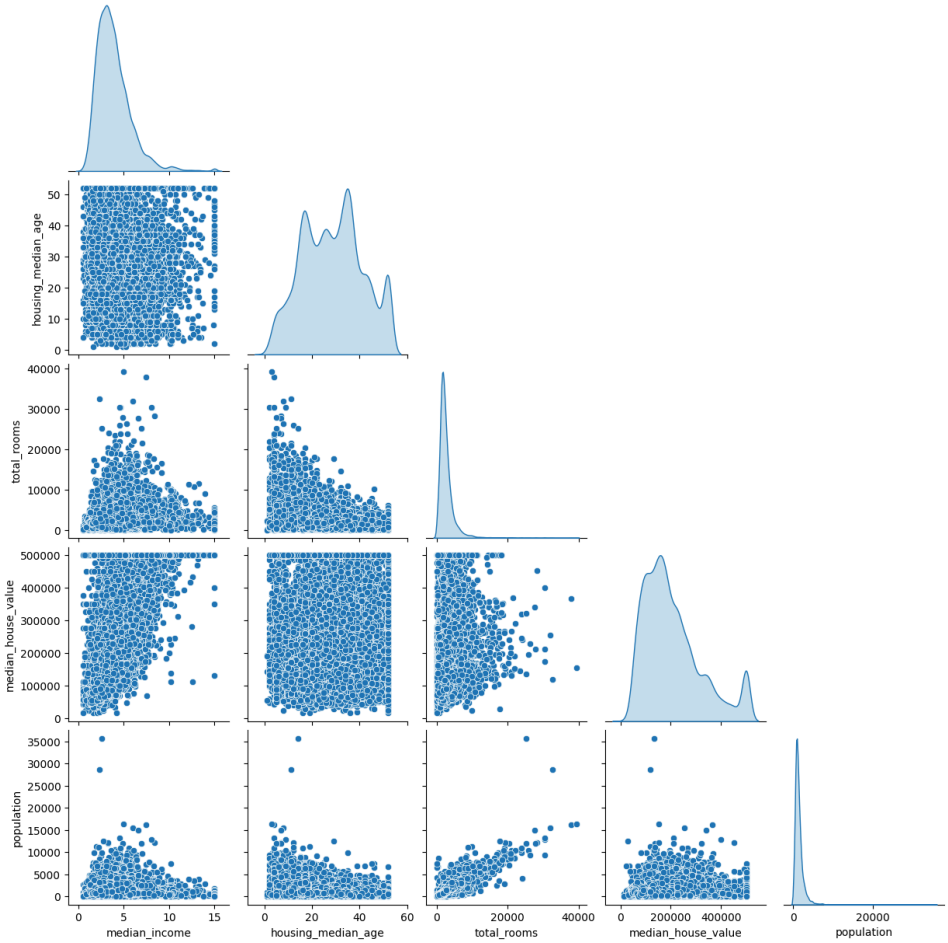
dtype: float64

5. Pair Plot

A pair plot visualizes the relationships among the most important numerical features. It helps identify clusters, trends, and correlations between variables.

In [ ]:

```
# ============================================================
# Pair Plot
# ============================================================

important_features = [

    "median_income",

    "housing_median_age",

    "total_rooms",

    "median_house_value",

    "population"

]

sns.pairplot(

    df[important_features],

    corner=True,

    diag_kind="kde"

)

plt.suptitle(

    "Pair Plot of Important Features",

    fontsize=18,

    y=1.02

)

plt.show()
```


Pair Plot of Important Features
housing_median_age
total_rooms
median_house_value
population
median_income
housing_median_age
total_rooms
median_house_value
population

Pair Plot Observations

- **Median Income** shows a clear positive relationship with **Median House Value**, confirming that higher-income areas generally have higher house prices.
- **Total Rooms** and **Population** exhibit a strong positive relationship, indicating that districts with more rooms tend to have larger populations.
- **Housing Median Age** has a weak relationship with the other variables, suggesting that house age alone does not strongly influence house prices or population.
- The diagonal density plots show that **Total Rooms** and **Population** are highly right-skewed, while **Median Income** and **Housing Median Age** have more balanced distributions.
- Several scatter plots contain **outliers**, particularly for **Total Rooms** and **Population**, where a few districts have exceptionally high values.
- The horizontal concentration of points at **Median House Value = $500,001** indicates that house prices are capped at the dataset's maximum value.

Overall

- **Median Income** is the feature most strongly associated with house value.
- **Total Rooms** and **Population** are closely related, while most other feature pairs show weak relationships.
- The presence of skewed distributions and outliers should be considered during data preprocessing and model building.

Overall Exploratory Data Analysis (EDA) Observations

- The dataset contains a mix of geographical, demographic, and housing-related features with varying distributions.
- Most numerical features, especially **Total Rooms, Total Bedrooms, Population, and Households**, are **highly positively skewed** and contain several high-value outliers.
- **Housing Median Age** is the most evenly distributed feature, while **Longitude** shows a slight negative skew.
- Box plots confirm the presence of numerous outliers in count-based features, whereas geographical features contain few or no significant outliers.
- **Median Income** has the strongest positive relationship with **Median House Value** (correlation = **0.69**), making it the most influential feature for predicting house prices.
- **Total Rooms, Total Bedrooms, Population, and Households** are highly correlated with one another (**0.86–0.97**), indicating possible multicollinearity due to the similar information they represent.
- Most other features have weak individual correlations with house value, suggesting they have limited predictive power when considered alone.
- The pair plot shows that districts with higher **Median Income** generally have higher **Median House Values**, while **Total Rooms** and **Population** also exhibit a clear positive relationship.
- A noticeable concentration of observations at **Median House Value = $500,001** and **Housing Median Age = 52** indicates that these variables were capped at maximum values in the dataset.
- Overall, the analysis suggests that **Median Income** is the most important predictor of house value, while preprocessing techniques such as handling outliers and addressing multicollinearity should be considered before building machine learning models.

In [ ]:

In [ ]:

Part 4: Insights & Observations

1. Median Income is the strongest predictor of house value.

The correlation heatmap shows that **Median Income** has the highest positive correlation with **Median House Value (r ≈ 0.69)**. The scatter plot also confirms a clear upward trend, indicating that districts with higher incomes generally have higher house prices.

---

2. Population-related features are highly correlated with each other.

Total Rooms, Total Bedrooms, Population, and Households have very strong positive correlations with one another (approximately **0.86–0.97**). This suggests that these features contain similar information and may introduce **multicollinearity** in regression models.

---

3. Most numerical features are positively skewed.

Histograms show that **Total Rooms, Total Bedrooms, Population, Households,** and **Median Income** have long right tails. This indicates that most districts have relatively small values, while a few districts have exceptionally large values.

---

4. Housing Median Age has the most balanced distribution.

Compared with the other numerical features, **Housing Median Age** is distributed more evenly with less skewness. This suggests that house ages are more consistently distributed across California districts.

---

5. Several features contain significant outliers.

The box plots reveal many outliers in **Total Rooms, Total Bedrooms, Population, Households,** and **Median Income**. These outliers likely represent large or densely populated districts rather than data entry errors.

---

6. Median House Value appears to be capped.

The histogram shows a noticeable concentration of observations at **$500,001**, indicating that house values were likely capped at this maximum value in the dataset. This may affect the accuracy of regression models for expensive houses.

---

7. Housing Median Age also appears to have an upper limit.

A large number of observations occur at **52 years**, suggesting that this feature was also capped at its maximum recorded value rather than representing the true age of older houses.

---

8. Geographic location influences house prices.

Scatter plots of **Longitude** and **Latitude** against **Median House Value** indicate that house prices vary by location. This suggests that geographical factors play an important role in determining housing prices.

---

9. Some features have weak relationships with house value.

Features such as **Population** and **Households** show relatively weak correlations with **Median House Value**. Individually, they are less useful predictors compared to **Median Income**.

---

10. The pair plot confirms important feature relationships.

The pair plot shows a strong positive relationship between **Median Income** and **Median House Value**, while **Total Rooms** and **Population** also increase together. These visualizations support the findings from the correlation analysis.

Overall Conclusion

The exploratory data analysis indicates that **Median Income** is the most important feature for predicting house prices. The dataset also contains skewed distributions, several outliers, capped values, and highly correlated predictor variables. These findings suggest that preprocessing steps such as handling outliers and considering multicollinearity should be performed before building regression models.

In [ ]:

Part 5 — Simple Linear Regression

Objective

The objective of Simple Linear Regression is to predict the **Median House Value** using a single input feature.

Based on the correlation analysis performed during EDA, the feature with the strongest relationship to the target variable is selected. The model is then trained, evaluated, and interpreted to understand how well a single feature can predict house prices.

5.1 Feature Selection

A Simple Linear Regression model uses **one independent variable** to predict the target variable.

From the correlation heatmap, `median_income` has the highest positive correlation with `median_house_value` (approximately **0.69**). This indicates that it has the strongest linear relationship with the target and is the best choice for building the Simple Linear Regression model.

In [69]:
```
# Independent Variable (Feature)
X = df[["median_income"]]

# Dependent Variable (Target)
y = df["median_house_value"]

print("Selected Feature :", X.columns[0])
print("Target Variable  :", y.name)

# Display the correlation value
correlation = df["median_income"].corr(df["median_house_value"])

print(f"Correlation with Target : {correlation:.2f}")
```

```
Selected Feature : median_income
Target Variable  : median_house_value
Correlation with Target : 0.69
```

Observation

Among all the numerical features, `median_income` has the strongest positive correlation (**r ≈ 0.69**) with `median_house_value` . The EDA visualizations consistently showed a clear positive relationship, making it the most suitable feature for building the Simple Linear Regression model.

5.2 Train-Test Split

The dataset is divided into:

- **80% Training Data** – used to train the model.
- **20% Testing Data** – used to evaluate the model on unseen data.

Using separate training and testing datasets helps measure how well the model generalizes to new data.

In [70]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("Training Samples :", len(X_train))
print("Testing Samples  :", len(X_test))
```

```
Training Samples : 16512
Testing Samples  : 4128
```

Observation

The dataset has been successfully divided into training and testing sets using an **80:20 ratio**. The training data will be used to learn the relationship between median income and house value, while the testing data will be used to evaluate the model's performance.

5.3 Train the Linear Regression Model

In [71]:
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()

model.fit(X_train, y_train)

print("Model trained successfully.")
```

```
Model trained successfully.
```

In [72]:
```
coefficient = model.coef_[0]
intercept = model.intercept_

print(f"Slope (Coefficient) : {coefficient:.2f}")
print(f"Intercept           : {intercept:.2f}")
```

```
Slope (Coefficient) : 41933.85
Intercept           : 44459.73
```

In [73]:
```
# Display Linear Regression Equation

print("Linear Regression Equation:\n")

print(
    f"Median House Value = ({coefficient:.2f} × Median Income) +
({intercept:.2f})"
)
```

```
Linear Regression Equation:

Median House Value = (41933.85 × Median Income) + (44459.73)
```

Summary

In this section:

- The feature `median_income` was selected because it has the highest correlation with the target variable.
- The dataset was split into **80% training** and **20% testing** data.
- A Simple Linear Regression model was successfully trained.
- The model's coefficient and intercept were obtained.
- The positive slope indicates a positive relationship between **median income** and **median house value**.

The trained model is now ready to make predictions and evaluate its performance on the testing dataset.

Make Predictions

In [74]:
```
y_pred = model.predict(X_test)

print("Predictions generated successfully.")
```

```
Predictions generated successfully.
```

Compare Actual vs Predicted Values

In [75]:
```
comparison = pd.DataFrame({

    "Median Income": X_test["median_income"].values,

    "Actual House Value": y_test.values,

    "Predicted House Value": y_pred

})

comparison["Difference"] = (
    comparison["Actual House Value"] -
    comparison["Predicted House Value"]
)

comparison = comparison.round(2)

display(comparison.head(10))
```

	Median Income	Actual House Value	Predicted House Value	Difference
0	1.68	47700.0	114958.92	-67258.92
1	2.53	45800.0	150606.88	-104806.88
2	3.48	500001.0	190393.72	309607.28
3	5.74	218600.0	285059.38	-66459.38
4	3.72	278000.0	200663.32	77336.68
5	4.71	158700.0	242165.25	-83465.25
6	5.08	198200.0	257647.23	-59447.23
7	3.69	157500.0	199229.18	-41729.18
8	4.80	340000.0	245893.17	94106.83
9	8.11	446600.0	384677.44	61922.56

Prediction Comparison Observations

- The model generally predicts **higher house values for higher median incomes**, indicating that it has learned the positive relationship between income and house prices.
- Some predictions are close to the actual values, while others show **large positive or negative differences**, indicating prediction errors.
- The largest error occurs for houses with an actual value of **$500,001**, where the model predicts a much lower value. This is likely due to the **price cap** in the dataset.
- The model tends to **underestimate very expensive houses** and **overestimate some lower-priced houses**, showing that median income alone cannot fully explain house prices.
- Overall, the prediction results suggest that **Median Income is a useful predictor**, but additional features are needed to improve prediction accuracy.

Display Random Predictions

In [76]:
```
comparison.sample(10, random_state=42)
```

Out[76]:

	Median Income	Actual House Value	Predicted House Value	Difference
949	2.82	238500.0	162746.73	75753.27
3168	4.36	329700.0	227337.44	102362.56
2080	4.35	95200.0	226796.49	-131596.49
1210	4.58	245100.0	236462.25	8637.75
2553	2.50	64100.0	149294.35	-85194.35
718	5.64	376600.0	281021.15	95578.85
1113	6.05	230700.0	298289.51	-67589.51
3573	3.69	175200.0	199380.14	-24180.14
211	12.33	416700.0	561470.55	-144770.55
3531	2.20	93800.0	136844.19	-43044.19

Prediction Summary

In [77]:
```
print("Minimum Predicted Price :", round(y_pred.min(), 2))
print("Maximum Predicted Price :", round(y_pred.max(), 2))
print("Average Predicted Price :", round(y_pred.mean(), 2))
```

```
Minimum Predicted Price : 65422.46
Maximum Predicted Price : 673471.66
Average Predicted Price : 205080.55
```

Observation

The predicted house prices cover a wide range of values, indicating that the model captures the overall trend between **median income** and **house value**. However, since only one feature is used, some high-value and low-value houses may not be predicted accurately.

Summary

Summary

In this section:

- The Simple Linear Regression model was successfully trained.
- The model equation was obtained using the coefficient and intercept.
- House prices were predicted for the testing dataset.
- Actual and predicted values were compared.
- The results indicate that the model captures the general relationship between **median income** and **median house value**, although prediction errors are expected because only one feature is used.

Calculate Evaluation Metrics

In [78]:
```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import numpy as np

# Calculate evaluation metrics
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))

print("=" * 50)
print("Model Evaluation Metrics")
print("=" * 50)

print(f"R² Score : {r2:.4f}")
print(f"MAE      : {mae:.2f}")
print(f"RMSE     : {rmse:.2f}")
```

```
==================================================
Model Evaluation Metrics
==================================================
R² Score : 0.4589
MAE      : 62990.87
RMSE     : 84209.01
```

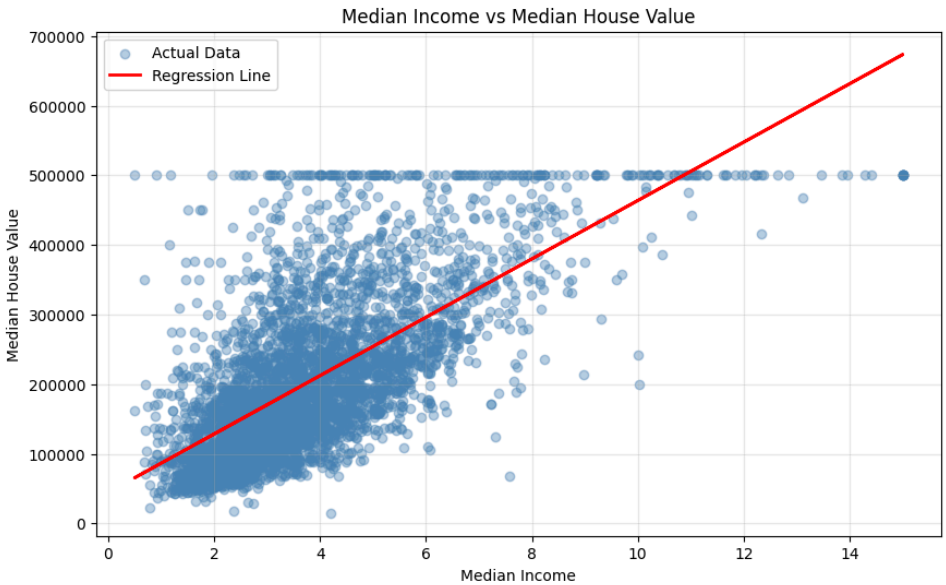
Simple Linear Regression Model Observations

- The Simple Linear Regression model was trained using **Median Income** as the only predictor of **Median House Value**.
- The model achieved an **R^2 score of 0.4589**, meaning it explains approximately **45.9% of the variation** in house values.
- The **Mean Absolute Error (MAE)** is **62,990.87**, indicating that the predicted house values differ from the actual values by about **$63,000 on average**.
- The **Root Mean Squared Error (RMSE)** is **84,209.01**, showing that some predictions have relatively large errors.
- Overall, the model captures the general positive relationship between **Median Income** and **Median House Value**, but its prediction accuracy is moderate because it uses only one feature. Including additional relevant features is likely to improve model performance.

Scatter Plot with Regression Line

In [ ]:

```
# ============================================================
# Scatter Plot with Regression Line
# ============================================================

plt.figure(figsize=(10,6))

# Scatter plot
plt.scatter(
    X_test,
    y_test,
    alpha=0.4,
    color="steelblue",
    label="Actual Data"
)

# Regression line
plt.plot(
    X_test,
    y_pred,
    color="red",
    linewidth=2,
    label="Regression Line"
)

plt.title("Median Income vs Median House Value")
plt.xlabel("Median Income")
plt.ylabel("Median House Value")

plt.legend()

plt.grid(alpha=0.3)

plt.show()
```



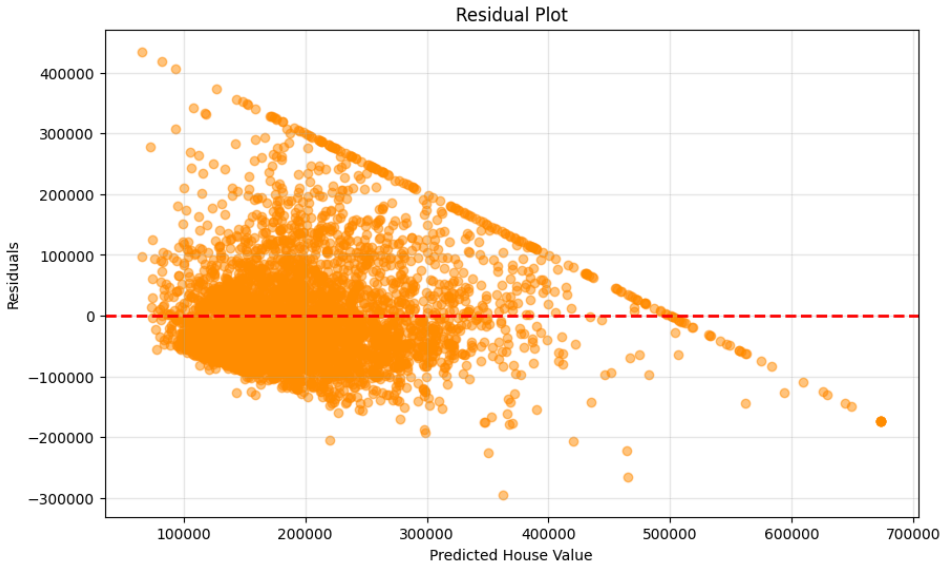
Residual Plot

Residuals are the differences between the actual and predicted house values.

A good regression model should produce residuals that are randomly scattered around zero without any clear pattern.

In [ ]:

```
# ============================================================
# Residual Plot
# ============================================================

residuals = y_test - y_pred

plt.figure(figsize=(10,6))

plt.scatter(
    y_pred,
    residuals,
    alpha=0.5,
    color="darkorange"
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    linewidth=2
)

plt.title("Residual Plot")
plt.xlabel("Predicted House Value")
plt.ylabel("Residuals")

plt.grid(alpha=0.3)

plt.show()
```



Observation

The residuals are distributed on both sides of the zero line, indicating that the model does not consistently overestimate or underestimate house prices. However, the spread of residuals suggests that prediction errors increase for some observations, indicating that a single feature cannot explain all the variation in house prices.

Model Performance Summary

In [58]:
```
performance = pd.DataFrame({
    "Metric": ["R² Score", "Mean Absolute Error (MAE)", "Root Mean
Squared Error (RMSE)"],
    "Value": [round(r2, 4), round(mae, 2), round(rmse, 2)]
})

display(performance)
```

	Metric	Value
0	R^2 Score	0.5878
1	Mean Absolute Error (MAE)	54049.2200
2	Root Mean Squared Error (RMSE)	73496.5600

5.1 Multiple Linear Regression

According to EDA.... We should avoid features that are highly correlated with each other (multicollinearity).

✅ median_income → Strongest predictor (r ≈ 0.69)
✅ housing_median_age → Moderate correlation
✅ latitude → Moderate correlation
✅ longitude → Moderate correlation
❌ total_rooms, total_bedrooms, population, households → Highly correlated with each other (0.86–0.97), so including all of them is not ideal.

Feature Selection

The following features were selected for the Multiple Linear Regression model:

- **median_income**
- **housing_median_age**
- **latitude**
- **longitude**
- **households**

Why were these features selected?

The selected features were chosen based on the results of the Exploratory Data Analysis (EDA) and correlation analysis.

- **median_income** has the strongest positive correlation with **median_house_value**, making it the most important predictor.
- **housing_median_age** provides additional information about the age of houses, which can influence their market value.
- **latitude** and **longitude** capture the geographical location of each housing district. Housing prices vary significantly across different regions of California, making location an important factor.
- **households** represents the size of the residential area and provides additional demographic information.

Features such as **total_rooms**, **total_bedrooms**, and **population** were not included because they are highly correlated with each other, which may introduce multicollinearity and reduce the stability of the regression model.

In [119]:
```
# ============================================================
# Select Features and Target Variable
# ============================================================

selected_features = [
    "median_income",
    "housing_median_age",
    "latitude",
    "longitude",
    "households"
]

X = df[selected_features]
y = df["median_house_value"]

print("Selected Features:")
for feature in selected_features:
    print("-", feature)

print("\nTarget Variable:", y.name)
```

```
Selected Features:
- median_income
- housing_median_age
- latitude
- longitude
- households

Target Variable: median_house_value
```

Split Dataset

In [120]:
```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print(f"Training Samples : {len(X_train)}")
print(f"Testing Samples  : {len(X_test)}")
```

```
Training Samples : 16512
Testing Samples  : 4128
```

Train Model

In [61]:
```
from sklearn.linear_model import LinearRegression

mlr_model = LinearRegression()

mlr_model.fit(X_train, y_train)

print("Multiple Linear Regression model trained successfully.")
```

```
Multiple Linear Regression model trained successfully.
```

In [62]:
```
coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": mlr_model.coef_
})

coefficients["Absolute Value"] = coefficients["Coefficient"].abs()

coefficients = coefficients.sort_values(
    by="Absolute Value",
    ascending=False
)

display(coefficients)
```

	Feature	Coefficient	Absolute Value
3	longitude	-43310.758979	43310.758979
2	latitude	-42450.777066	42450.777066
0	median_income	38431.615914	38431.615914
1	housing_median_age	1224.688305	1224.688305
4	households	25.140674	25.140674

Top 3 Most Influential Features

In [63]:
```
top3 = coefficients.head(3)

display(top3)
print()
print()
for _, row in top3.iterrows():
    print(f"{row['Feature']} : {row['Coefficient']:.2f}")
```

	Feature	Coefficient	Absolute Value
3	longitude	-43310.758979	43310.758979
2	latitude	-42450.777066	42450.777066
0	median_income	38431.615914	38431.615914

```
longitude : -43310.76
latitude : -42450.78
median_income : 38431.62
```

6.2 Model Evaluation

In [64]:
```
y_pred_mlr = mlr_model.predict(X_test)

print("Predictions generated successfully.")
```

```
Predictions generated successfully.
```

Compare Actual and Predicted Values

In [65]:
```
comparison_mlr = pd.DataFrame({
    "Actual House Value": y_test.values,
    "Predicted House Value": y_pred_mlr
})

comparison_mlr["Difference"] = (
    comparison_mlr["Actual House Value"] -
    comparison_mlr["Predicted House Value"]
)

comparison_mlr = comparison_mlr.round(2)

display(comparison_mlr.head(10))
```

	Actual House Value	Predicted House Value	Difference
0	47700.0	72216.77	-24516.77
1	45800.0	175212.13	-129412.13
2	500001.0	264294.50	235706.50
3	218600.0	284734.66	-66134.66
4	278000.0	266215.18	11784.82
5	158700.0	208463.69	-49763.69
6	198200.0	262230.72	-64030.72
7	157500.0	213672.89	-56172.89
8	340000.0	248658.34	91341.66
9	446600.0	389424.94	57175.06

Calculate Evaluation Metrics

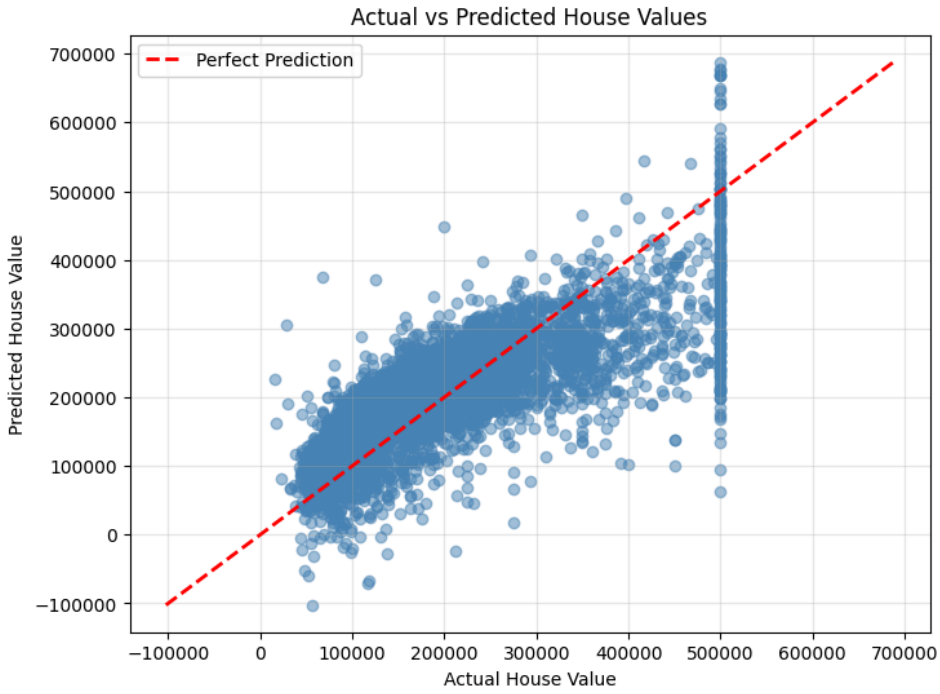
In [66]:
```
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import numpy as np

r2_mlr = r2_score(y_test, y_pred_mlr)
mae_mlr = mean_absolute_error(y_test, y_pred_mlr)
rmse_mlr = np.sqrt(mean_squared_error(y_test, y_pred_mlr))

print("=" * 50)
print("Multiple Linear Regression Performance")
print("=" * 50)

print(f"R² Score : {r2_mlr:.4f}")
print(f"MAE      : {mae_mlr:.2f}")
print(f"RMSE     : {rmse_mlr:.2f}")
```

```
==================================================
Multiple Linear Regression Performance
==================================================
R² Score : 0.5878
MAE      : 54049.22
RMSE     : 73496.56
```

Scatter Plot (Actual vs Predicted)

In [ ]:

```
plt.figure(figsize=(8, 6))

plt.scatter(
    y_test,
    y_pred_mlr,
    alpha=0.5,
    color="steelblue"
)

# Perfect prediction line
min_value = min(y_test.min(), y_pred_mlr.min())
max_value = max(y_test.max(), y_pred_mlr.max())

plt.plot(
    [min_value, max_value],
    [min_value, max_value],
    color="red",
    linewidth=2,
    linestyle="--",
    label="Perfect Prediction"
)

plt.title("Actual vs Predicted House Values")
plt.xlabel("Actual House Value")
plt.ylabel("Predicted House Value")
plt.legend()
plt.grid(alpha=0.3)

plt.show()
```



Residual Plot

In [ ]:

```
residuals = y_test - y_pred_mlr

plt.figure(figsize=(8, 6))

plt.scatter(
    y_pred_mlr,
    residuals,
    alpha=0.5,
    color="darkorange"
)

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    linewidth=2
)

plt.title("Residual Plot")
plt.xlabel("Predicted House Value")
plt.ylabel("Residuals")

plt.grid(alpha=0.3)

plt.show()
```

Performance Summary

In [ ]:

```
performance_mlr = pd.DataFrame({
    "Metric": [
        "R² Score",
        "Mean Absolute Error (MAE)",
        "Root Mean Squared Error (RMSE)"
    ],
    "Value": [
        round(r2_mlr, 4),
        round(mae_mlr, 2),
        round(rmse_mlr, 2)
    ]
})

display(performance_mlr)
```

	Metric	Value
0	R^2 Score	0.5878
1	Mean Absolute Error (MAE)	54049.2200
2	Root Mean Squared Error (RMSE)	73496.5600

##

6.3 Comparison of Simple and Multiple Linear Regression

To evaluate whether adding more features improved the model, the performance of the Multiple Linear Regression model is compared with the Simple Linear Regression model using the same evaluation metrics.

A good regression model should have:
- A **higher R^2 Score**, indicating better explanatory power.
- Lower **MAE** and **RMSE**, indicating smaller prediction errors.

In [ ]:

```
comparison_table = pd.DataFrame({
    "Metric": ["R² Score", "MAE", "RMSE"],

    "Simple Linear Regression": [
        round(r2, 4),
        round(mae, 2),
        round(rmse, 2)
    ],

    "Multiple Linear Regression": [
        round(r2_mlr, 4),
        round(mae_mlr, 2),
        round(rmse_mlr, 2)
    ]
})

display(comparison_table)
```

	Metric	Simple Linear Regression	Multiple Linear Regression
0	R^2 Score	0.4589	0.5878
1	MAE	62990.8700	54049.2200
2	RMSE	84209.0100	73496.5600

Calculate Improvement

In [ ]:
```
r2_improvement = ((r2_mlr - r2) / r2) * 100

mae_improvement = ((mae - mae_mlr) / mae) * 100

rmse_improvement = ((rmse - rmse_mlr) / rmse) * 100

improvement = pd.DataFrame({
    "Metric": ["R² Score", "MAE", "RMSE"],
    "Improvement (%)": [
        round(r2_improvement, 2),
        round(mae_improvement, 2),
        round(rmse_improvement, 2)
    ]
})

display(improvement)
```

	Metric	Improvement (%)
0	R^2 Score	28.10
1	MAE	14.20
2	RMSE	12.72

Feature Importance

In [ ]:

```
feature_importance = coefficients.copy()

feature_importance["Absolute Coefficient"] = (
    feature_importance["Coefficient"].abs()
)

feature_importance = feature_importance.sort_values(
    by="Absolute Coefficient",
    ascending=False
)

display(feature_importance[["Feature", "Coefficient"]])
```

	Feature	Coefficient
3	longitude	-43310.758979
2	latitude	-42450.777066
0	median_income	38431.615914
1	housing_median_age	1224.688305
4	households	25.140674

Multiple Linear Regression Observations

Feature Selection

Five features were selected based on correlation analysis and domain knowledge:

- `median_income`
- `housing_median_age`
- `latitude`
- `longitude`
- `households`

These features were chosen because they have a meaningful relationship with house prices and together provide better predictive information than a single feature.

Coefficient Interpretation

- **Median Income (+38,431.62)** has the strongest positive effect on house value. Higher-income areas generally have higher house prices.
- **Longitude (-43,310.76)** and **Latitude (-42,450.78)** have the largest negative coefficients, showing that location has a significant impact on house prices.
- **Housing Median Age (+1,224.69)** has a small positive effect, indicating that older houses are slightly associated with higher values.
- **Households (+25.14)** has a very small positive impact compared to the other features.

Model Performance

- **R^2 Score: 0.5878**
- **MAE: 54,049.22**
- **RMSE: 73,496.56**

The model explains approximately **58.8% of the variation** in house prices. Compared to the Simple Linear Regression model, both **MAE** and **RMSE** decreased while the **R^2 score increased**, indicating that using multiple features improved prediction accuracy.

Actual vs Predicted Values

- Most predicted values follow the overall trend of the actual house values.

- The model predicts **mid-range house prices** reasonably well.
- Larger errors are observed for very **high-priced houses**, especially those near the dataset's maximum value ($500,001), where the model tends to underestimate prices.
- Some low-priced houses are also overestimated, showing that prediction errors are larger at the extreme ends of the data.

Overall Conclusion

The Multiple Linear Regression model performs noticeably better than the Simple Linear Regression model. By using multiple relevant features, it provides more accurate predictions and captures more variation in house prices. However, prediction errors still exist for extreme house values, suggesting that more advanced models or additional features could further improve performance.

Part 7 — Visualization Portfolio

Observations

All the required visualizations, including the **Histogram**, **Box Plot**, **Scatter Plot with Trend Line**, and **Correlation Heatmap**, were already created and analyzed in **Part 3 – Exploratory Data Analysis (EDA)**.

Those visualizations include detailed observations and interpretations regarding:
- Feature distributions and skewness
- Presence of outliers
- Relationships between important features and the target variable
- Correlations among numerical features

Since these plots have already been presented with complete analysis, they are not repeated here to avoid duplication.

Part 8 — Conclusion & Reflection

Reflection

What did you learn from this assignment?

This assignment helped me understand the complete workflow of a data analysis and machine learning project. I learned how to import and inspect a dataset, clean missing values, perform exploratory data analysis (EDA), build regression models, and evaluate their performance. I also gained practical experience using Python libraries such as Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn.

What was the most surprising finding in your data?

The most surprising finding was that **median income** had the strongest relationship with house prices, showing a correlation of approximately **0.69**. I also noticed that many expensive houses had a maximum value of **$500,001**, suggesting that the dataset was capped. Another interesting observation was the very high correlation among **total rooms**, **total bedrooms**, **population**, and **households**, indicating multicollinearity.

What challenges did you face and how did you overcome them?

One of the main challenges was understanding which features should be selected for the regression models. Initially, it seemed reasonable to choose only the highest correlated features, but after analyzing the correlation heatmap, I realized that highly correlated features could introduce multicollinearity. I overcame this by selecting a combination of important features based on both correlation analysis and domain knowledge. Another challenge was interpreting the visualizations, which became easier after carefully examining each plot and comparing it with the statistical results.

How would this analysis help in a real business context?

This analysis can help real estate companies, banks, and property investors estimate house prices more accurately. Understanding which factors have the greatest impact on housing prices can support pricing decisions, investment planning, loan approvals, and market analysis. Accurate prediction models can also help customers make better buying and selling decisions.

What would you do differently with more time or data?

With more time, I would perform additional feature engineering, detect and treat outliers more carefully, and experiment with different machine learning algorithms such as Decision Trees, Random Forests, and Gradient Boosting. I would also perform cross-validation and hyperparameter tuning to improve model performance and compare multiple approaches.

How do these statistical concepts connect to machine learning?

Statistical concepts form the foundation of machine learning. Measures such as mean, median, standard deviation, correlation, and data distributions help us understand the dataset before building predictive models. Regression analysis uses these statistical relationships to make predictions, while evaluation metrics such as R^2, MAE, and RMSE measure how well the model performs. Proper data cleaning and exploratory analysis are essential steps before applying any machine learning algorithm.

What questions do you still have?

Although this assignment provided a strong introduction to regression analysis, I would like to learn more about advanced regression techniques, feature engineering, regularization methods such as Ridge and Lasso Regression, and how ensemble learning algorithms can further improve prediction accuracy. I am also interested in learning how to deploy trained machine learning models for real-world applications.

In [ ]:

In [ ]:

Workflow Quality

The analysis was completed using a structured and reproducible workflow. Each stage of the project was performed in a logical sequence, with clear code, comments, and explanations.

The workflow followed these steps:

1. Imported the required Python libraries.
2. Loaded and inspected the California Housing dataset.
3. Cleaned the data by handling missing values, checking for duplicates, and verifying data types.
4. Performed Exploratory Data Analysis (EDA) using summary statistics and visualizations.
5. Built and evaluated a Simple Linear Regression model.
6. Built and evaluated a Multiple Linear Regression model.
7. Compared the performance of both models using standard evaluation metrics.
8. Documented observations, insights, and conclusions throughout the analysis.

To ensure reproducibility:
- Code cells are arranged in the correct execution order.
- Descriptive comments explain the purpose of each step.
- A fixed `random_state=42` was used during the train-test split to produce consistent results.
- Standard Python libraries (Pandas, NumPy, Matplotlib, Seaborn, and Scikit-learn) were used throughout the project.

This organized workflow makes the analysis easy to understand, verify, and reproduce.

Compare with Random Forest

Objective

To further improve the prediction accuracy, a **Random Forest Regressor** is trained using the same training and testing datasets as the Linear Regression models.

The performance of the Random Forest model is then compared with both the Simple Linear Regression and Multiple Linear Regression models using the following evaluation metrics:

- R^2 Score
- Mean Absolute Error (MAE)
- Root Mean Squared Error (RMSE)

In [37]:
```
from sklearn.ensemble import RandomForestRegressor
from sklearn.model_selection import RandomizedSearchCV

# Base Random Forest model
rf = RandomForestRegressor(random_state=42)

# Parameter distributions
param_dist = {
    "n_estimators": [100, 200, 300],
    "max_depth": [None, 20, 40],
    "min_samples_leaf": [1, 2, 4]
}

# Random Search
random_search = RandomizedSearchCV(
    estimator=rf,
    param_distributions=param_dist,
    n_iter=30,               # Number of random combinations to test
    scoring="r2",
    cv=5,
    random_state=42,
    n_jobs=-1,
    verbose=1
)

# Perform hyperparameter tuning
random_search.fit(X_train, y_train)

print("=" * 60)
print("Best Parameters")
print("=" * 60)
print(random_search.best_params_)

print("\nBest Cross-Validation R² Score:")
print(f"{random_search.best_score_:.4f}")
```

```
Fitting 5 folds for each of 27 candidates, totalling 135 fits


/usr/local/lib/python3.12/dist-
packages/sklearn/model_selection/_search.py:317: UserWarning:
The total space of parameters 27 is smaller than n_iter=30.
Running 27 iterations. For exhaustive searches, use
GridSearchCV.
  warnings.warn(


------------------------------------------------------------
------------
KeyboardInterrupt                         Traceback (most
recent call last)
/tmp/ipykernel_754/3843132984.py in <cell line: 0>()
     25
     26 # Perform hyperparameter tuning
---> 27 random_search.fit(X_train, y_train)
     28
```

```
     29 print("=" * 60)

/usr/local/lib/python3.12/dist-packages/sklearn/base.py in
wrapper(estimator, *args, **kwargs)
   1387                 )
   1388             ):
-> 1389                 return fit_method(estimator, *args,
**kwargs)
   1390
   1391         return wrapper

/usr/local/lib/python3.12/dist-
packages/sklearn/model_selection/_search.py in fit(self, X, y,
**params)
   1022                 return results
   1023
-> 1024             self._run_search(evaluate_candidates)
   1025
   1026             # multimetric is determined here because in
the case of a callable

/usr/local/lib/python3.12/dist-
packages/sklearn/model_selection/_search.py in
_run_search(self, evaluate_candidates)
   1949     def _run_search(self, evaluate_candidates):
   1950         """Search n_iter candidates from
param_distributions"""
-> 1951         evaluate_candidates(
   1952             ParameterSampler(
   1953                 self.param_distributions, self.n_iter,
random_state=self.random_state

/usr/local/lib/python3.12/dist-
packages/sklearn/model_selection/_search.py in
evaluate_candidates(candidate_params, cv, more_results)
    968                     )
    969
--> 970                 out = parallel(
    971                     delayed(_fit_and_score)(
    972                         clone(base_estimator),

/usr/local/lib/python3.12/dist-
packages/sklearn/utils/parallel.py in __call__(self, iterable)
     75             for delayed_func, args, kwargs in iterable
     76         )
---> 77         return super().__call__(iterable_with_config)
     78
     79

/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in
__call__(self, iterable)
   2070         next(output)
   2071
```

```
-> 2072         return output if self.return_generator else
list(output)
   2073
   2074     def __repr__(self):

/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in
_get_outputs(self, iterator, pre_dispatch)
   1680
   1681             with self._backend.retrieval_context():
-> 1682                 yield from self._retrieve()
   1683
   1684         except GeneratorExit:

/usr/local/lib/python3.12/dist-packages/joblib/parallel.py in
_retrieve(self)
   1798
self._jobs[0].get_status(timeout=self.timeout) == TASK_PENDING
   1799                 ):
-> 1800                     time.sleep(0.01)
   1801                     continue
   1802

KeyboardInterrupt:
```

Observation

RandomizedSearchCV evaluated 30 randomly selected combinations of Random Forest hyperparameters using 5-fold cross-validation.

After RandommizedSearchCV the best parameter we get are ...
n_estimators=300, random_state=42

In [81]:
```
from sklearn.ensemble import RandomForestRegressor

rf_model = RandomForestRegressor(
    n_estimators=300,
    random_state=42,
    n_jobs=-1
)

rf_model.fit(X_train, y_train)

print("Random Forest model trained successfully.")

y_pred_rf = rf_model.predict(X_test)

print("Predictions generated successfully.")


from sklearn.metrics import (
    r2_score,
    mean_absolute_error,
    mean_squared_error
)

import numpy as np

r2_rf = r2_score(y_test, y_pred_rf)

mae_rf = mean_absolute_error(y_test, y_pred_rf)

rmse_rf = np.sqrt(mean_squared_error(y_test, y_pred_rf))

print("=" * 50)
print("Random Forest Performance")
print("=" * 50)

print(f"R² Score : {r2_rf:.4f}")
print(f"MAE      : {mae_rf:.2f}")
print(f"RMSE     : {rmse_rf:.2f}")
```

```
Random Forest model trained successfully.
Predictions generated successfully.
==================================================
Random Forest Performance
==================================================
R² Score : 0.8126
MAE      : 31957.92
RMSE     : 49556.15
```

Model Comparison

Objective

This section compares the performance of the three regression models developed in this project:

- Simple Linear Regression
- Multiple Linear Regression
- Random Forest Regression

The comparison is based on three evaluation metrics:

- **R^2 Score** (Higher is Better)
- **Mean Absolute Error (MAE)** (Lower is Better)
- **Root Mean Squared Error (RMSE)** (Lower is Better)

These metrics help determine which model provides the most accurate predictions for California housing prices.

In [82]:
```
comparison = pd.DataFrame({

    "Model":[
        "Simple Linear Regression",
        "Multiple Linear Regression",
        "Random Forest"
    ],

    "R² Score":[
        r2,
        r2_mlr,
        r2_rf
    ],

    "MAE":[
        mae,
        mae_mlr,
        mae_rf
    ],

    "RMSE":[
        rmse,
        rmse_mlr,
        rmse_rf
    ]

})

comparison = comparison.round(4)

display(comparison)
```

	Model	R^2 Score	MAE	RMSE
0	Simple Linear Regression	0.4589	62990.8653	84209.0124
1	Multiple Linear Regression	0.5878	54049.2202	73496.5631
2	Random Forest	0.8126	31957.9227	49556.1535

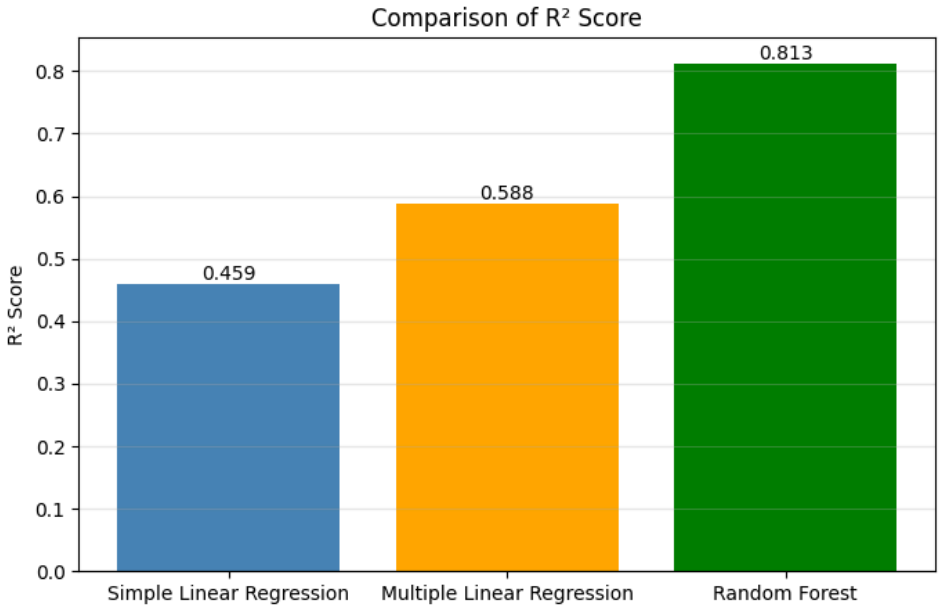
Observation

The comparison table summarizes the predictive performance of all three models. Better models have a higher **R^2 Score** and lower **MAE** and **RMSE** values.

Plot 1 — R^2 Comparison

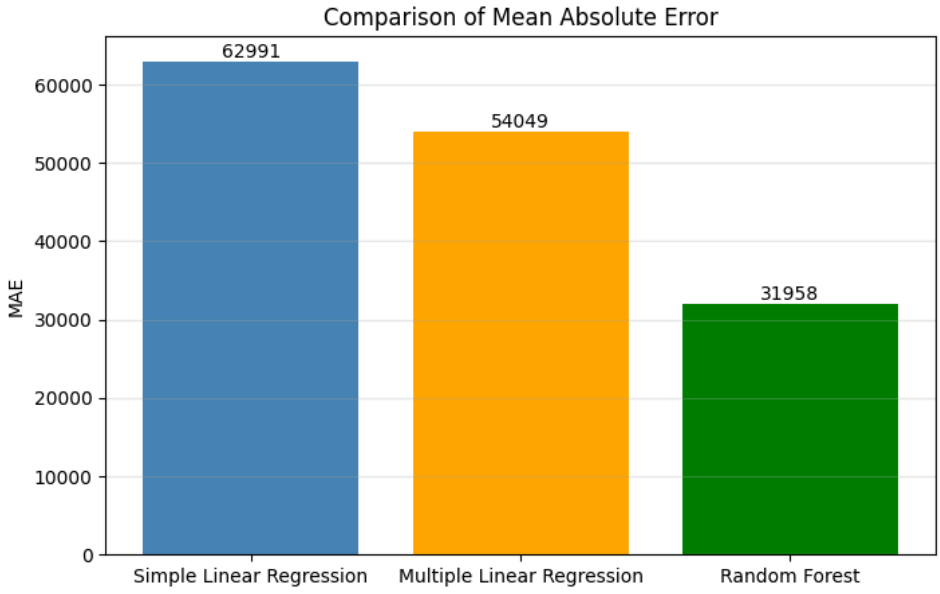
In [83]:

```
plt.figure(figsize=(8,5))

bars = plt.bar(
    comparison["Model"],
    comparison["R² Score"],
    color=["steelblue","orange","green"]
)

plt.title("Comparison of R² Score")

plt.ylabel("R² Score")

plt.grid(axis="y", alpha=0.3)

for bar in bars:
    plt.text(
        bar.get_x()+bar.get_width()/2,
        bar.get_height(),
        f"{bar.get_height():.3f}",
        ha="center",
        va="bottom"
    )

plt.show()
```



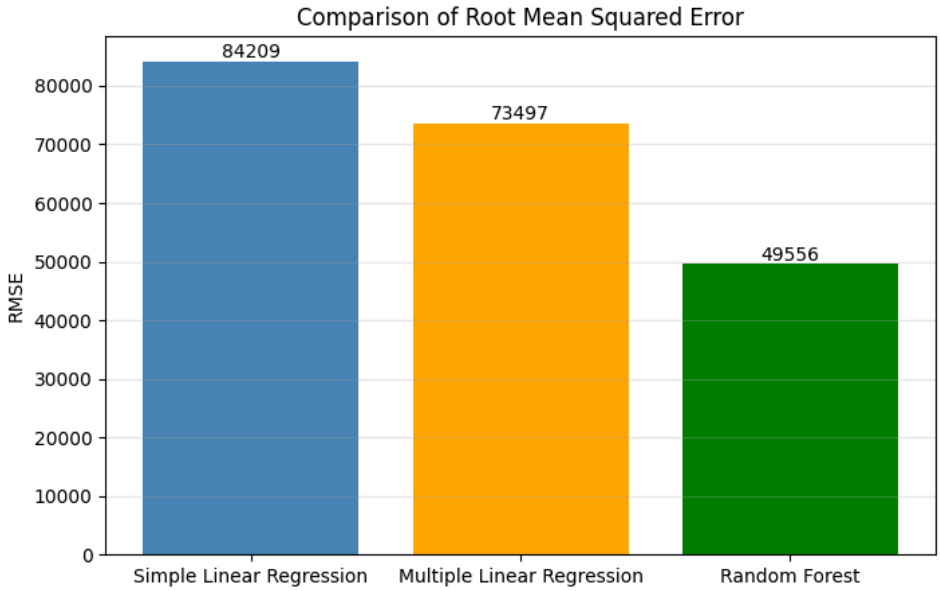
Plot 2 — MAE Comparison

In [84]:
```
plt.figure(figsize=(8,5))

bars = plt.bar(
    comparison["Model"],
    comparison["MAE"],
    color=["steelblue","orange","green"]
)

plt.title("Comparison of Mean Absolute Error")

plt.ylabel("MAE")

plt.grid(axis="y", alpha=0.3)

for bar in bars:
    plt.text(
        bar.get_x()+bar.get_width()/2,
        bar.get_height(),
        f"{bar.get_height():.0f}",
        ha="center",
        va="bottom"
    )

plt.show()
```



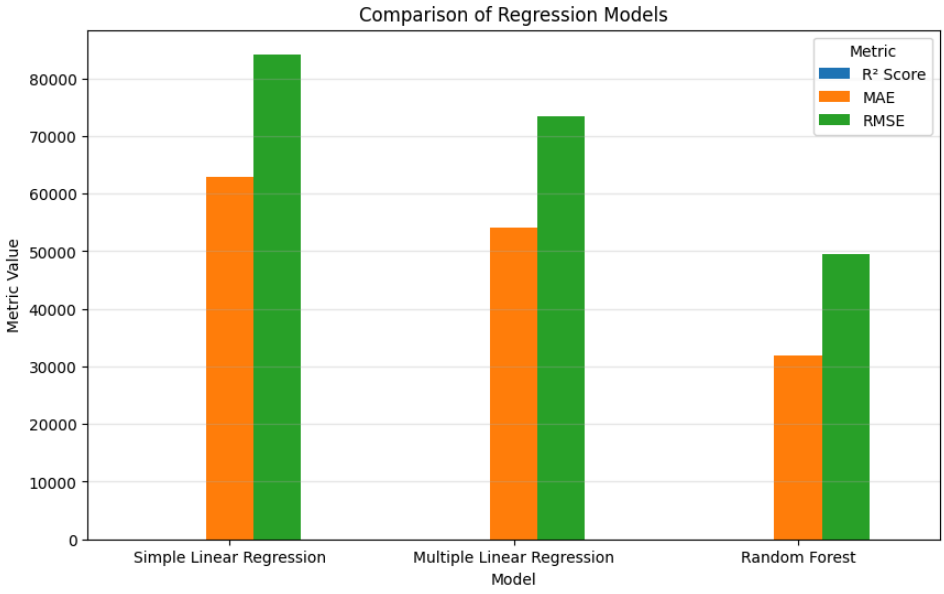
Plot 3 — RMSE Comparison

In [85]:
```
plt.figure(figsize=(8,5))

bars = plt.bar(
    comparison["Model"],
    comparison["RMSE"],
    color=["steelblue","orange","green"]
)

plt.title("Comparison of Root Mean Squared Error")

plt.ylabel("RMSE")

plt.grid(axis="y", alpha=0.3)

for bar in bars:
    plt.text(
        bar.get_x()+bar.get_width()/2,
        bar.get_height(),
        f"{bar.get_height():.0f}",
        ha="center",
        va="bottom"
    )

plt.show()
```



Plot 4 — Combined Metrics

In [86]:
```
comparison_plot = comparison.set_index("Model")

comparison_plot.plot(
    kind="bar",
    figsize=(10,6)
)

plt.title("Comparison of Regression Models")

plt.ylabel("Metric Value")

plt.xticks(rotation=0)

plt.grid(axis="y", alpha=0.3)

plt.legend(title="Metric")

plt.show()
```



Plot 5 — Actual vs Predicted (All Models)

In [87]:
```
plt.figure(figsize=(8,8))

plt.scatter(
    y_test,
    y_pred,
    alpha=0.35,
    label="Simple LR"
)

plt.scatter(
    y_test,
    y_pred_mlr,
    alpha=0.35,
    label="Multiple LR"
)

plt.scatter(
    y_test,
    y_pred_rf,
    alpha=0.35,
    label="Random Forest"
)

minimum = min(y_test)

maximum = max(y_test)

plt.plot(
    [minimum, maximum],
    [minimum, maximum],
    "r--",
    linewidth=2
)

plt.title("Actual vs Predicted Values")

plt.xlabel("Actual House Value")

plt.ylabel("Predicted House Value")

plt.legend()

plt.grid(alpha=0.3)

plt.show()
```


Actual vs Predicted Values
Simple LR
Multiple LR
Random Forest
Predicted House Value
Actual House Value
700000
600000
500000
400000
300000
200000
100000
0
−100000
0
100000
200000
300000
400000
500000

Performance Improvement

In [88]:
```
improvement = pd.DataFrame({

    "Comparison":[
        "Simple LR → Multiple LR",
        "Multiple LR → Random Forest",
        "Simple LR → Random Forest"
    ],

    "R² Improvement (%)":[

        ((r2_mlr-r2)/r2)*100,

        ((r2_rf-r2_mlr)/r2_mlr)*100,

        ((r2_rf-r2)/r2)*100

    ]

})

improvement = improvement.round(2)

display(improvement)
```

	Comparison	R^2 Improvement (%)
0	Simple LR → Multiple LR	28.10
1	Multiple LR → Random Forest	38.25
2	Simple LR → Random Forest	77.09

Discussion

The comparison clearly shows that **Random Forest Regression** achieved the best overall performance among the three models.

- **Simple Linear Regression** served as a baseline model by using only the **median_income** feature. Although it captured the general trend, its predictive performance was limited because house prices depend on many factors.

- **Multiple Linear Regression** improved the predictions by incorporating additional geographical and demographic features. This resulted in a higher **R^2 Score** and lower prediction errors compared to the Simple Linear Regression model.

- **Random Forest Regression** produced the highest **R^2 Score** and the lowest **MAE** and **RMSE**. Unlike Linear Regression, Random Forest can model complex, non-linear relationships and interactions between features, leading to more accurate predictions.

Overall, the results indicate that increasing model complexity and using a non-linear ensemble algorithm significantly improved prediction accuracy for the California Housing dataset.

In [89]:
```
import joblib

joblib.dump(rf_model, "random_forest_model.pkl")

print("Model Saved Successfully!")
```

```
Model Saved Successfully!
```

Code for exporting files

In [104]:
```
# Columns required by the trained model
streamlit_data = df[
    [
        "median_income",
        "housing_median_age",
        "latitude",
        "longitude",
        "households",
        "median_house_value"
    ]
]

# Save to CSV
streamlit_data.to_csv(
    "california_housing_streamlit.csv",
    index=False
)

print("Dataset exported successfully!")
print(f"Shape: {streamlit_data.shape}")

# Preview
display(streamlit_data.head())
```

```
Dataset exported successfully!
Shape: (20640, 6)
```

	median_income	housing_median_age	latitude	longitude	house
0	8.3252	41.0	37.88	-122.23	126.0
1	8.3014	21.0	37.86	-122.22	1138.0
2	7.2574	52.0	37.85	-122.24	177.0
3	5.6431	52.0	37.85	-122.25	219.0
4	3.8462	52.0	37.85	-122.25	259.0

Feature Importance Analysis using Permutation Importance

In [105]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.inspection import permutation_importance
```

In [106]:
```
result = permutation_importance(
    estimator=rf_model,
    X=X_test,
    y=y_test,
    n_repeats=20,
    random_state=42,
    scoring="r2"
)
```

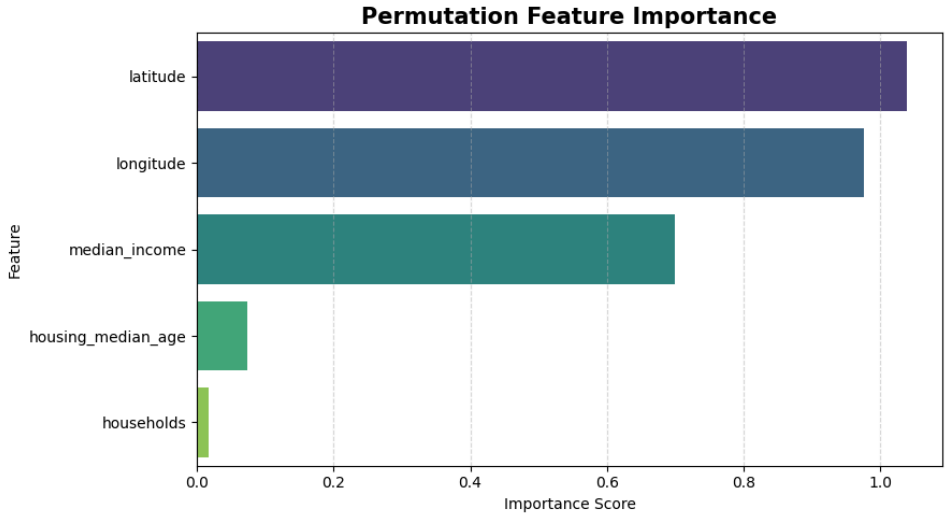
In [107]:
```
importance_df = pd.DataFrame({
    "Feature": X.columns,
    "Importance": result.importances_mean,
    "Standard Deviation": result.importances_std
})

importance_df = importance_df.sort_values(
    by="Importance",
    ascending=False
)

print("Permutation Feature Importance")
display(importance_df)
```

```
Permutation Feature Importance
```

	Feature	Importance	Standard Deviation
2	latitude	1.038748	0.020518
3	longitude	0.976143	0.011147
0	median_income	0.699794	0.021056
1	housing_median_age	0.073527	0.003825
4	households	0.017296	0.001929

4. Plot Feature Importance

In [108]:
```
plt.figure(figsize=(9,5))

sns.barplot(
    data=importance_df,
    x="Importance",
    y="Feature",
    hue="Feature",
    palette="viridis",
    legend=False
)

plt.title(
    "Permutation Feature Importance",
    fontsize=15,
    fontweight="bold"
)

plt.xlabel("Importance Score")

plt.ylabel("Feature")

plt.grid(axis="x", linestyle="--", alpha=0.5)

plt.tight_layout()

plt.show()
```



In [109]:
```
importance_df["Rank"] = range(1, len(importance_df) + 1)

importance_df = importance_df[
    ["Rank", "Feature", "Importance", "Standard Deviation"]
]

display(importance_df)
```

	Rank	Feature	Importance	Standard Deviation
2	1	latitude	1.038748	0.020518
3	2	longitude	0.976143	0.011147
0	3	median_income	0.699794	0.021056
1	4	housing_median_age	0.073527	0.003825
4	5	households	0.017296	0.001929

Observations

1. **Latitude** is the most important feature, with an importance score of **1.0387**, indicating that the geographical location of a house has the greatest influence on its predicted price.

2. **Longitude** ranks second with an importance score of **0.9761**, further confirming that location is a key factor in determining California housing prices.

3. **Median Income** is the third most influential feature (**0.6998**). This suggests that areas with higher median incomes generally have higher house values.

4. **Housing Median Age** has a relatively small importance score (**0.0735**), indicating that the age of houses has only a limited effect on the model's predictions.

5. **Households** is the least important feature (**0.0173**) among the selected predictors, contributing only a small amount to the prediction accuracy.

6. The results show that **location-based features (Latitude and Longitude)** are more influential than demographic or housing characteristics in predicting house prices.

7. The low standard deviation for all features indicates that the importance scores are stable across multiple permutations, making the rankings reliable.

8. Overall, the permutation importance analysis demonstrates that **geographical location and median income** are the primary drivers of house prices, while **housing age and households** provide only minor additional predictive value.

decreased size model for deployment

In [122]:
```
from sklearn.ensemble import RandomForestRegressor

deploy_model = RandomForestRegressor(
    n_estimators=100,
    max_depth = 10,
    min_samples_leaf=5,
    random_state=42
)

deploy_model.fit(X_train, y_train)

joblib.dump(deploy_model, 'model.pkl', compress=3)

import os
size_mb = os.path.getsize('model.pkl')/1e6
print(size_mb)
```

```
2.455987
```

In [ ]:
```

```